# IT584: Approximation Algorithms

## Project Report

Research Paper: *A Batch-to-Online Transformation under Random-Order Model*

*by Jing Dong and Yuichi Yoshida*


Group Members:

- Dharmik Godhani 202201311
- Heer Dharsandiya 202203059
- Mit Mehta 202203002
- Yash Tarpara 202201422
- Yuvraj Kathad 202203013

*Dhirubhai Ambani University*

# Contents


**1 Problem** **2**

**2 Preliminaries** **2**
Notations . . . 2
Offline Learning . . . 2
Online Learning with Random-Order Model . . . 2
Regret . . . 2
Average sensitivity . . . 3

**3 Batch-to-Online Transformation in the Random-Order Model** **3**

**4 Approximation Algorithm with Low Average Sensitivity via Coreset** **6**
4.1 Sensitivity-Based Coreset Construction . . . 7
4.2 Main Sensitivity Bound . . . 8

**5 Application of the framework** **8**
Online $(k, z)$-clustering . . . 8
5.1 Online $(k, z)$-clustering . . . 8
Online Low-Rank Matrix Approximation . . . 11
5.2 Online Low-Rank Matrix Approximation . . . 12
Online regression . . . 14
5.3 Online regression . . . 15

**6 Experiments** **18**

**7 Conclusion** **19**

# 1 Problem

The paper gives a framework that can be used to develop online algorithms with low $\epsilon$-approximate regret in the random-order model from offline approximation algorithms. The paper also demonstrates that offline approximation algorithms can be transformed into a low-sensitivity version using a coreset construction method. This framework is then applied on three problems: online $(k, z)$-clustering, online matrix approximation, and online regression.

# 2 Preliminaries

## Notations

- $[n]$ denotes $\{1, 2, \ldots, n\}$.
- For $a, b \in \mathbb{R}$, $a \in (1 \pm \epsilon)b$ is the short hand for $(1 - \epsilon)b \leq a \leq (1 + \epsilon)b$.

## Offline Learning

The offline learning problem is that given a set of $n$ data points, $X$, we are supposed to learn the parameter $\theta$, such that the value of the objective function $\ell(X, \theta) := \sum_{x \in X} \ell(x, \theta)$ is minimized.

## Online Learning with Random-Order Model

In the online learning problem, instead of receiving all points at once, the data arrives sequentially throughout a time horizon $n$. The data point $x_t$ comes at time $t \in [n]$. At the time $t$, we have the data points $X_{t-1} := (x_1, \ldots, x_{t-1})$ and using that we output a parameter $\theta_t$. Then we receive the data point $x_t$ and incur a loss of $\ell(\theta_t, x_t)$.

In random-order model, the data points $x_1, \ldots, x_n$ may be chosen adversarially, but their ordering is randomly permuted before the algorithm runs.

## Regret

Regret is a way to measure how well an online algorithm performs compared to the best possible solution of the offline algorithm. It is calculated by taking the difference of total loss incurred in solution of online algorithm and total loss incurred in solution of offline algorithm. But if finding the exact solution for the offline problem is hard, then we use $\epsilon$-approximate regret.

The $\epsilon$-approximate regret of an algorithm for the random-order model is defined as:

$$Reg_\epsilon(n) = \mathbb{E}_{\mathcal{A},\{x_t\}}\Big[\sum_{t=1}^{n} \ell(\theta_t, x_t) - (1+\epsilon)\sum_{t=1}^{n} \ell(\theta^*, x_t)\Big] \tag{1}$$

where $\theta^* = \arg\min_{\theta} \sum_{t=1}^{n} \ell(\theta, x_t)$ is the exact solution of the offline algorithm.

### Average sensitivity

Average sensitivity is a way to measure how much a randomized algorithm's output can change if you remove just one point from the dataset.

How much the output changes is described by Total Variation(TV). We define the total variation distance between two probability distributions, $P$ and $Q$, as:

$$TV(P, Q) := \sup_{A} \Big|P(A) - Q(A)\Big|.$$

Let $\mathcal{A}$ be a (randomized) algorithm that takes an input $X$ and outputs $\mathcal{A}(X)$. For $\beta : \mathbb{Z}^+ \to \mathbb{R}^+$, we say that the average sensitivity of $\mathcal{A}$ is at most $\beta$ if:

$$\frac{1}{n}\sum_{i=1}^{n} TV(\mathcal{A}(X), \mathcal{A}(X^{(i)})) \leq \beta(n) \tag{2}$$

where $X^{(i)} = (x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_n)$, i.e. the set obtained by deleting the $i^{th}$ data point from $X$, and $\mathcal{A}(X)$, $\mathcal{A}(X^{(i)})$ are the output probability distributions.

## 3 Batch-to-Online Transformation in the Random-Order Model

Our goal is to create a general framework that can transform any offline $(1+\epsilon)$-approximate algorithm into an online algorithm with low $\epsilon$ regret i.e Eq. 1. For this we have:

**Theorem 1.** *Let $\mathcal{A}$ be a (randomized) $(1+\epsilon)$-approximate algorithm for the offline learning algorithm with average sensitivity $\beta : \mathbb{Z}^+ \to \mathbb{R}^+$. Then, there exists an online learning algorithm in the random-order model such that*

$$Regret_\epsilon(n) = O\left(\sum_{t=1}^{n} \beta(t) + 1\right).$$

To understand our method we have:

**Algorithm 1** General batch-to-online conversion

**Require:** Offline approximation algorithm $\mathcal{A}$.

1: **for** $t = 1, \ldots, n$ **do**
2: Obtain $\theta_t$ by running $\mathcal{A}$ on $X_{t-1}$.
3: Receive $x_t$ and $\ell(\theta_t, x_t)$.

To prove that Algorithm 1 achieves a low approximate regret when $\mathcal{A}$ has a low average sensitivity, we use:

**Lemma 1.** *Let $\mathcal{A}$ be a (randomized) algorithm for the offline learning problem with average sensitivity $\beta : \mathbb{Z}^+ \to \mathbb{R}^+$. Then for any input of $n$ points, $X$, we have*

$$\frac{1}{n}\sum_{i=1}^{n} \mathbb{E}_{\mathcal{A}}\left[\ell(\mathcal{A}(X^{(i)}), x_i)\right] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}_{\mathcal{A}}\left[\ell(\mathcal{A}(X), x_i)\right] \pm \beta(n),$$

*where $x = a \pm b$ means $a - b \leq x \leq a + b$.*

**Proof intuition*(Lemma 1)*:**
If an algorithm is stable (has low average sensitivity $\beta(n)$), then its average performance on unseen data (approximated by leave-one-out error) is very close to its average performance on the full training data.

- Let $a = \mathbb{E}_{\mathcal{A}}\left[\ell(\mathcal{A}(X^{(i)}), x_i)\right]$ and $b = \mathbb{E}_{\mathcal{A}}\left[\ell(\mathcal{A}(X), x_i)\right]$.
- We know that, $a = b + (a - b)$.
- Taking sum and average on both side: $\frac{1}{n}\sum a = \frac{1}{n}\sum b + \frac{1}{n}\sum(a - b)$
- Since $x \leq |x|$, we have $a - b \leq |a - b| = |b - a|$
- Substituting the values and using the concept of average sensitivity from Eq.2 we will get the final result.

**Proof intuition*(Theorem 1)***

- In online learning, we make a prediction $\theta_t$ using past data $X_{t-1}$ and suffer a loss on the next point $x_t$. Ideally, the loss on this new point $\ell(\theta_t, x_t)$ shouldn't be much worse than the average loss the same model $\theta_t$ would get on the data it was trained on $X_{t-1}$.

- When is the loss on the next point $x_t$ close to the average loss on the past points $X_{t-1}$? It happens when the algorithm is stable (Low average sensitivity $\beta$).

- The loss on the next point $x_{t+1}$ using model $\theta_{t+1} = \mathcal{A}(X_t)$ is close to the average training loss of $\theta_{t+1}$ on $X_t$. The difference is controlled by the sensitivity $\beta(t)$ (specifically $2\beta(t)$ after the analysis).

$$\mathbb{E}_{\mathcal{A},\{x_i\}}\left[\ell(\theta_{t+1}, x_{t+1}) - \frac{1}{t}\ell(\theta_{t+1}, X_t)\right] \leq 2\beta(t)$$

- Rearranging the terms, we have

$$\mathbb{E}_{\mathcal{A},\{x_i\}}\left[\ell(\theta_{t+1}, x_{t+1})\right] \leq \mathbb{E}_{\mathcal{A},\{x_i\}}\left[\frac{\ell(\theta_{t+1}, X_t)}{t}\right] + 2\beta(t) \leq \mathbb{E}_{\{x_i\}}\left[\frac{(1+\epsilon)\mathrm{OPT}_t}{t}\right] + 2\beta(t),$$

- The algorithm isn't perfect even on its training data. It has an approximation ratio $(1+\epsilon)$ compared to the true optimum $OPT_t$ on the training set $X_t$.

- We relate $OPT_t$ and $OPT_n$ by showing that

$$\mathbb{E}_{\{x_i\}}\left[\sum_{t=1}^{n} \frac{\mathrm{OPT}_t}{t}\right] \leq OPT_n$$

- The total online loss:

$$\mathbb{E}_{\mathcal{A},\{x_i\}}\left[\sum_{t=1}^{n} \ell(\theta_t, x_t)\right] = \mathbb{E}_{\mathcal{A},\{x_i\}}\left[\ell(\theta_1, x_1)\right] + \mathbb{E}_{\mathcal{A},\{x_i\}}\left[\sum_{t=1}^{n-1} \ell(\theta_{t+1}, x_{t+1})\right]$$

$$\leq 1 + \mathbb{E}_{\{x_i\}}\left[\sum_{t=1}^{n-1} \frac{(1+\epsilon)\mathrm{OPT}_t}{t}\right] + 2\sum_{t=1}^{n-1} \beta(t).$$

- Hence we have,

$$\mathbb{E}_{\mathcal{A},\{x_i\}}\left[\sum_{t=1}^{n} \ell(\theta_t, x_t)\right] - (1+\epsilon)\mathrm{OPT}_n = O\left(\sum_{t=1}^{n} \beta(t) + 1\right).$$

# 4 Approximation Algorithm with Low Average Sensitivity via Coreset

In this section, we present a general method for transforming any offline approximation algorithm into one whose average sensitivity is small. The idea is to first compress the dataset into a *coreset*—a small, weighted subset of the dataset.

If you have an offline algorithm that is stable, then you can turn it into a good online algorithm that performs well in the random-order model. But most offline algorithms aren't naturally stable. So how do we build a stable offline algorithm?

Instead of trying to make our offline algorithm stable, let's first create a stable summary of the data called a **coreset**. Then, we run our usual (possibly unstable) offline algorithm on this small, stable summary.

## What is a Coreset?

A coreset is a small, weighted sample of the dataset that preserves key properties. For example, if you have a dataset of a million images, a coreset might be just 1,000 carefully chosen and weighted images. Running on this smaller set gives nearly the same result as on the full dataset.

**Definition 1.** *A weighted set $(Y, w)$ is an $\epsilon$-coreset for dataset $X$ if, for any solution $\theta$, the total loss on $(Y, w)$ is within a factor of $(1 \pm \epsilon)$ of the total loss on $X$.*

$$\sum_{y \in Y} w(y)\, \ell(\theta, y) \;\in\; (1 \pm \epsilon) \sum_{x \in X} \ell(\theta, x).$$

To build such a coreset, we sample points according to their *sensitivity*, which measures the maximum fraction of total loss one point can command.

For each $x \in X$, we define its sensitivity as

$$\sigma_X(x) \;=\; \sup_{\theta} \frac{\ell(\theta, x)}{\ell(\theta, X)}.$$

**Analogy:** Coreset can be thought of as the summary of a long book (the input data). You don't have time to read the entire book, so you rely on a much shorter summary that captures its main ideas. While writing such a summary, you'll find that some sentences describe the book better than others—those sentences are more "important" and are more likely to be included.

## 4.1 Sensitivity-Based Coreset Construction

We adopt the classical importance-sampling routine with a minor deviation to stabilize weights:

**Algorithm 2** Coreset via Sensitivity Sampling

**Require:** Loss function $\ell$, dataset $X$, $m \in \mathbb{N}$, accuracy parameter $\epsilon > 0$

1: For each $x \in X$, compute $\sigma_X(x)$ and set $p(x) = \sigma_X(x) / \sum_{x'} \sigma_X(x')$
2: Initialize $S$ be an empty set.
3: **for** $i = 1$ to $m$ **do**
4: Sample $x$ with probability $p(x)$
5: Sample $\tilde{p}$ from $[p(x),\, (1+\epsilon)\, p(x)]$ uniformly at random.
6: **if** $w(x)$ is undefined **then**
7: $S \leftarrow Sx$
8: $w(x) \leftarrow 1/\tilde{p}$
9: **else**
10: $w(x) \leftarrow w(x) + 1/\tilde{p}$
11: **return** $(S, w)$

**Explanation *(Algorithm 2)*:**

- For every single data point x in the large dataset X, calculate its sensitivity $\sigma_X(x)$. Recall that $\sigma_X(x)$ measures the maximum relative contribution of point x to the total loss, across all possible models . It tells you how "important" a point can be.

- Create a probability distribution p(x) over all points in X by normalizing the sensitivities. $p(x) = \sigma_X(x)/(\text{Sum of all sensitivities})$. Points with higher sensitivity get a higher probability of being selected later.

- For $m$ times, randomly select a point x from the original dataset X according to the probability distribution $p(x)$ calculated in Step 1. Points with higher sensitivity $\sigma_X(x)$ are more likely to be picked in this step.

- For the specific point $x$, choose a random value $\tilde{p}$ uniformly from the small interval $[p(x), (1+\epsilon)p(x)]$. This slight complication provides some theoretical slack needed in proofs, connecting the process directly to the target accuracy $\epsilon$.

- If point $x$ is not already in the coreset, add it to the coreset and set its weight to $1/\tilde{p}$. Else just add $1/\tilde{p}$ to its weight. This is called Inverse Probability Weighting, its a standard technique in importance sampling. Points with low sensitivity have low probability $p(x)$ of being sampled. If we do happen to sample such a point, it needs to represent itself and all the other similar low-sensitivity points that weren't sampled. Giving it a large weight $1/\tilde{p}$ achieves this.

- By taking $m$ samples, we reduce the randomness of the construction process and increase the likelihood that the final weighted set $(S, w)$ is a good $\epsilon$-approximation of $X$ for all models $\theta$.

## 4.2 Main Sensitivity Bound

Our goal is to show that the sampling procedure itself has low average sensitivity.

**Lemma 2.** *The average sensitivity of Algorithm 2 satisfies*

$$\frac{1}{n}\sum_{i=1}^{n} \mathrm{TV}\big(\mathcal{D}_X,\ \mathcal{D}_{X^{(i)}}\big) = O\Big(\tfrac{m}{\epsilon\, n}\Big),$$

*where $\mathcal{D}_X$ is the distribution over coresets when sampling from $X$.*

*Proof Intuition of Lemma 2.* We give a high-level intuition in three simple steps.

- **Only one cluster moves:** The data is partitioned into $m$ clusters. Removing any single point affects only its own cluster's sampling probabilities; the other $m-1$ clusters remain unchanged.
- **Average over $n$ deletions:** Since there are $n$ possible deletions and each deletion can disturb at most one cluster's worth of mass, the *average* disturbance per deletion is
$$\frac{1 \text{ cluster}}{n \text{ deletions}} \approx \frac{m}{n},$$
because the total mass of all $m$ clusters is $m$.
- **Jitter from random weights:** Algorithm 2 chooses each weight uniformly in a $(1+\epsilon)$-wide interval rather than exactly proportional to its ideal sensitivity. This "jitter" can amplify any small change by up to a factor of $\frac{1}{\epsilon}$.

Putting these together, the average sensitivity satisfies

$$(\text{cluster shift}) \ \times\ (\text{weight jitter}) \ =\ O\Big(\tfrac{m}{n}\Big) \ \times\ O\Big(\tfrac{1}{\epsilon}\Big) \ =\ O\Big(\tfrac{m}{\epsilon\, n}\Big),$$

as claimed. □

# 5 Application of the framework

## 5.1 Online $(k, z)$-clustering

### Problem Setup

Assume we have a big cloud of data points say $X$ in a $d$-dimensional space. The goal is to group this points into exactly $k$ clusters i.e. $Z = \{z_1, z_2, ..., z_k\}$ .We want to

minimize the total cost, where the cost for a single data point $x$ is the distance to its closest center in $Z$, raised to the power of $z$, i.e. $\ell(Z_t, x_t) := \min_{j=1,\ldots,k} \|x_t - z_j\|_2^z$. In the online version of the problem, as we are about to discuss here, instead of seeing all the data points ($X$) at once, they arrive one by one, *i.e.*, $x_t$ at time $t$. Before you see the next point $x_t$, you must decide on the best set of $k$ cluster centers $Z_t$ based only on the points seen so far, $x_1, \ldots, x_{t-1}$. When $x_t$ arrives, you calculate the cost using the centers $Z_t$ you just committed to:

$$\ell(Z_t, x_t) = \left\| x_t - \arg\min_{z \in Z_t} \|x_t - z\|_2 \right\|_2^z$$

The specific order in which the points $x_1, \ldots, x_n$ arrive is assumed to be a uniformly random permutation of the full dataset.

$z$ determines how distance is penalized.

- $z = 1$ (k-median): Minimize the sum of actual distances. Good for finding centers in dense regions.
- $z = 2$ (k-means): Minimize the sum of squared distances. Heavily penalizes outliers, centers tend to be averages.
- Other $z \geq 1$ values offer different trade-offs.

We can think of this as sorting a huge pile of LEGO bricks ($X$) into $k$ bins and $z$ changes how badly you want to penlize putting a brick of wrongly matched color in a bin ($z_n$).

### Objective

Minimize the $\epsilon$-approximate regret. We compare the total cost accumulated by our online algorithm $\sum \ell(Z_t, x_t)$ to $(1+\epsilon)$ times the cost of the single best fixed set of $k$ centers that could have been chosen if we knew all the data from the start, denoted as $\mathrm{OPT}_n$. We want our online cost to be very close to this optimal solution.

### Algorithms

**Algorithm 3** Online consistent $(k, z)$-clustering

**Require:** PTAS algorithm $\mathcal{D}$ for $(k, z)$-clustering, approximation ratio $\epsilon, \delta \in (0, 1)$.

```
for t = 1, ..., n do
    Construct an ε-coreset C_{t-1} = (S_{t-1}, ω_{t-1}) by running Algorithm 7 on X_{t-1}.
    Obtain cluster set Z_t by running a PTAS D with approximation ratio of (1+ε) on C_{t-1}.
    Receive x_t and ℓ(Z_t, x_t).
```

**Algorithm 4** Coreset construction for clustering Huang and Vishnoi [2020]

**Require:** A set of point $X$, approximation parameter $\epsilon, \delta \in (0,1)$, integer $k, z$.

Set $\epsilon = \epsilon/c$ for some large constant $c > 0$.

Compute a $k$-center set $C_t^* \subseteq \mathbb{R}^d$ as an $\epsilon$-approximation of the $(k,z)$-clustering problem over $X$ with the $D^z$-sampling algorithm with an approximation ratio of $O(2^z \log k)$.

**for** each $x \in X$ **do**

  Compute the closest point to $x$ in $C^*$, $c^*(x)$, with ties broken arbitrarily.

  For each $c \in C^*$, denote $X^c$ to be the set of points $x \in X$ with $c^*(x) = c$.

**for** each $x \in X$ **do**

  Set $\sigma_{1,X}(x) = 2^{2z+2}\epsilon^2 \left( \frac{\|x - c^*(x)\|^z}{\sum_{x \in X} \ell(C^*, x)} + \frac{1}{|X^{c^*(x)}|} \right)$.

Pick a non-uniform random sample $D^1$ of $N_1 = O((168z)^{10z}\epsilon^{-5z-15}k^5 \log \frac{k}{\delta})$ points from $X$, where $x \in X$ is selected with probability $\frac{\sigma_{1,X}(x)}{\sum_{y \in X} \sigma_{1,X}(y)}$.

**for** each $x \in D^1$ **do**

  Sample $\tilde{u}_X(x)$ from $\left[ \frac{\sum_{y \in X} \sigma_{1,X}(y)}{|D^1| \cdot \sigma_{1,X}(x)}, (1+\epsilon) \frac{\sum_{y \in X} \sigma_{1,X}(y)}{|D^1| \cdot \sigma_{1,X}(x)} \right]$.

  Set $u_X(x) = \tilde{u}_X(x)$.

**for** each $c \in C^*$ **do**

  Compute $D_c$ to be the set of points in $D^1$ whose closest point in $C^*$ is $c$.

**for** each $x \in D^1$ **do**

  Set $\sigma_{2,X}(x) = \frac{u_X(x) \cdot \ell(C^*, x)}{\sum_{y \in D^1} u_X(y) \cdot \ell(C^*, y)}$.

Pick a non-uniform random sample $D^2$ of $N_2 = O(\epsilon^{-2z-2} k \log k \log \frac{k}{\epsilon\delta})$ points from $X_t$, where $x \in X_t$ is selected with probability $\frac{\sigma_{2,X}(x)}{\sum_{y \in D^1} \sigma_{2,X}(y)}$.

**for** each $x \in D^2$ **do**

  Sample $\tilde{w}_X(x)$ from $\left[ \frac{\sum_{y \in D^1} \sigma_{2,X}(y)}{|D^2| \cdot \sigma_{2,X}(x)}, (1+\epsilon) \frac{\sum_{y \in D^1} \sigma_{2,X}(y)}{|D^2| \cdot \sigma_{2,X}(x)} \right]$.

  Set $w_X(x) = \tilde{w}_X(x)$.

**for** each $c \in C^*$ **do**

  Set $w_X(c) = (1 + 10\epsilon) \sum_{x \in D_c} u_X(x) - \sum_{x \in D^2 \cap D_c} w_X(x)$.

Set $S = D^2 \cup C^*$

Set $w(x) = w_X(x)$

**return** $(S, w)$.

**Explanation (*Algorithm 3*):**

At each time step $t$ (from 1 to $n$):

1. Call Coreset Construction Algorithm to generate an $\epsilon'$-coreset representing all the data points received previously.

2. Run an offline PTAS (Polynomial Time Approximation Scheme) $\mathcal{D}$ for $(k,z)$-clustering on the constructed coreset.

3. Output the $k$ cluster centers $Z_t$ returned by the PTAS $\mathcal{D}$.
4. Receive the next data point and calculate the loss $\ell(Z_t, x_t)$.

**Explanation (*Algorithm 4*):**

1. **Find Initial Approximate Centers ($C^*$):** Find a rough approximate set of cluster centers $C^*$ for the full dataset $X$ using a fast method.
2. **First Sampling (Get $D^1$):**
   - Calculate an *"importance score"* for each point in $X$ (points farther from their nearest center in $C^*$ are more important).
   - Randomly sample a medium-sized set $D^1$ from $X$, giving higher sampling probability to important points.
   - Assign initial, slightly randomized weights to the sampled points.
3. **Second Sampling (Get $D^2$ from $D^1$):**
   - Re-calculate "importance scores" for each point in $D^1$, based on their weighted contribution to the clustering cost relative to $C^*$.
   - Randomly sample a smaller set $D^2$ from $D^1$, again using importance sampling.
   - Assign final, slightly randomized weights to the sampled points.
4. **Final Coreset:**
   - Combine the points from $D^2$ and the rough centers $C^*$ into a final set $S$.
   - Assign final weights: points in $D^2$ use their assigned weights; points in $C^*$ are assigned special weights to balance contributions from dropped points.

   **Result:** The final weighted set $(S, w)$ serves as a compact coreset. Clustering $S$ is much faster and gives results similar to clustering the full dataset $X$.

**Theorem 2.** *For any $\epsilon \in (0,1)$, Algorithm 4 has regret*

$$\mathit{Regret}_\epsilon(n) \leq O\left((168z)^{10z}\epsilon^{-5z-15}k^5 \log\frac{kn}{\epsilon} + \epsilon^{-2z-2}k\log k \log\frac{kn}{\epsilon}\log n\right)$$

## 5.2 Online Low-Rank Matrix Approximation

### Problem Setup

**Low-rank matrix approximation:** Say we have a big data matrix $A$ of size $d \times n$. We can think of $d$ as the number of features and $n$ as the number of data points. We want to find a smaller, simpler data matrix, $B$, that is still a good representation of the original big data matrix $A$. "Simpler" here means it has a low rank, which basically means it captures the most important patterns in the data without all the noise and details.

**Why the need for low-rank matrices?** For the same dimension, low-rank matrices are better than high-rank matrices because:

- Low-rank matrices when dealing with real-world data and applications like machine learning, data compression, and signal processing.
- Faster computation of operations like multiplication, inversion, eigenvalue decomposition, etc.
- Requires significantly less storage space.

If rank of matrix $A$ is $r$, then we need to find a matrix $B$ which has rank $k < r$. It can be found using Singular Value Decomposition of matrix $A$. Singular Value Decomposition method gives matrix $A$ as a product of three matrices:

$$A = U\Sigma V^\top$$

where

- $U$ is orthonormal matrix of size $d \times r$.
- $\Sigma$ is diagonal matrix of size $r \times r$. Its entries are singular values of matrix $A$, i.e., $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r$.
- $V$ is orthonormal matrix of size $r \times n$.

Now, we keep only the top $k$ singular values and their corresponding vectors:

$$A_k = U_k \Sigma_k V_k^\top$$

where $\Sigma_k$ is a diagonal matrix of size $k \times k$ with $A$'s top $k$ singular values on the diagonal, and $U_k$ (of size $d \times k$) and $V_k$ (of size $k \times n$) are orthonormal matrices obtained from $U$ and $V$, respectively, by gathering corresponding columns.

The matrix, $A_k$, so obtained is the best $k$-rank approximation of $A$, i.e.,

$$A_k = \underset{B \in \mathbb{R}^{d \times n},\ \mathrm{rank}(B) \leq k}{\mathrm{argmin}} \|A - B\|_F$$

where $\|P\|_F = \sqrt{\sum_{i=1}^{n}\sum_{j=1}^{m}|P_{ij}|^2}$ for an $n \times m$ matrix $P$.

The best rank-k approximation can also be found by projecting $A$ onto the span of its top $k$ singular vectors, that is, $A_k = U_k U_k^\top A$. Then, we can say an orthonormal matrix $Z$ is an $\epsilon$-approximate solution if

$$\|A - ZZ^\top A\|_F \leq (1+\epsilon)\|A - U_k U_k^\top A\|_F$$

where $Z$ is some orthonormal basis of dimension $k$.

**Online matrix approximation:** Through a time horizon of $n$, we receive a column of $A$, $a_t \in \mathbb{R}^d$ at each time step $t$. We are supposed to compute $Z_t \in \mathbb{R}^{d\times k}$ that minimizes

$$\ell(Z_t, a_t) = \|a_t - Z_t Z_t^\top a_t\|_F$$

### Method and results

In the context of low-rank matrix approximation, the coreset of a matrix is called the projection-cost preserving samples, defined as follows:

**Definition 2** (Rank-$k$ Projection-Cost Preserving Sample)**.** *For $n' < n$, a subset of rescaled columns $C \in \mathbb{R}^{d\times n'}$ of $A \in \mathbb{R}^{d\times n}$ is a $(1+\epsilon)$ projection-cost preserving sample if, for all rank-$k$ orthogonal projection matrices $X \in \mathbb{R}^{d\times d}$,*

$$(1-\epsilon)\|A - XA\|_F^2 \leq \|C - XC\|_F^2 \leq (1+\epsilon)\|A - XA\|_F^2.$$

Such a matrix $C$ can be constructed using importance sampling methods using ridge leverage score.

**Definition 3** (Ridge leverage score)**.** *For $i^{th}$ column $a_i$ of matrix $A$, the ridge leverage score is defined as:*

$$\tau_i(A) = \mathbf{a}_i^\top \left(AA^\top + \frac{\|A - A_k\|_F^2}{k} I\right)^\dagger \mathbf{a}_i$$

*where $\dagger$ denotes the Moore-Penrose pseudoinverse of a matrix.*

Given below is the online matrix approximation algorithm, which uses our transformation framework. It computes the approximation of the matrix from the projection-cost preserving sample (matrix $C$) derived from the matrix $A$ using ridge leverage scores.

**Algorithm 5** Online low rank matrix approximation

**Require:** Approximation parameters $\epsilon \in (0, 1)$.

Set $\delta = O(\frac{\epsilon}{n})$ and $m = O(\frac{k}{\epsilon^2} \log(\frac{k}{\delta}))$.
**for** $t = 1, \ldots, n$ **do**
    Construct $\mathbf{A}_{t-1} \in \mathbb{R}^{d \times (t-1)}$ by concatenating $a_1, \ldots, a_{t-1}$.
    Let $C_{t-1} \in \mathbb{R}^{d \times m}$ be the zero matrix.
    **for** $j = 1, \ldots, m$ **do**
        Sample $i^{th}$ column $a_i \in \mathbb{R}^d$ of $A_{t-1}$ with probability $p_i := \frac{\tau_i(\mathbf{A}_{t-1})}{\sum_{j'=1}^{t-1} \tau_{j'}(\mathbf{A}_{t-1})}$.
        Sample $w \in \mathbb{R}$ uniformly from $[\frac{1}{\sqrt{tp_i}}, \frac{(1+\epsilon)}{\sqrt{tp_i}}]$.
        Replace the $j$-th column of $C_{t-1}$ with $w \cdot a_i$.
    Set $Z_t \in \mathbb{R}^{d \times k}$ to the top $k$ left singular vectors of $C_{t-1}$.
    Receive $a_t \in \mathbb{R}^d$ and $\ell(Z_t, a_t) \in \mathbb{R}_+$.

**Explanation *(Algorithm 5)*:**

- We have $\epsilon \in (0, 1)$ as input.
- We set $\delta = O(\frac{\epsilon}{n})$ and $m = O(\frac{k}{\epsilon^2} \log(\frac{k}{\delta}))$.
- For each new column that comes in, the algorithm looks at all the columns it has seen so far and creates a coreset.
- We only need to include the columns which are "important", in the coreset. The ridge leverage scores are used to figure out how important a column is.
- The algorithm then randomly samples some of the most important columns to put into its coreset. It's more likely to pick the columns that are deemed more important by the leverage scores.
- When it picks a column for the coreset, it also rescales it slightly with a random number.
- Once the coreset is obtained, $Z_t$ is obtained from it using singular value decomposition.
- The algorithm does this process every time a new column of the matrix arrives. The loss $\ell(Z_t, a_t)$ is calculated each time.

**Theorem 3.** *For any $\epsilon \in (0, 1)$, Algorithm 5 has regret-*

$$\mathit{Regret}_\epsilon(n) = \mathcal{O}\left(\frac{k}{\epsilon^2} \log n \log\left(\frac{kn}{\epsilon}\right)\right).$$

## 5.3 Online regression

### Problem Statement

We consider the following online regression task. Over the time $t = 1, 2, \ldots, n$:

1. At time $t$, we choose a weight vector $\mathbf{x}_t \in \mathbb{R}^d$.
2. Then an example $(\mathbf{a}_t, b_t)$ arrives, where $\mathbf{a}_t \in \mathbb{R}^d$ is a feature vector and $b_t \in \mathbb{R}$ is the target.
3. The learner incurs the squared-error loss

$$\ell(\mathbf{x}_t;\, \mathbf{a}_t, b_t) \;=\; \left\|\mathbf{a}_t^\top \mathbf{x}_t - b_t\right\|_2.$$

   Without loss of generality, we assume that the losses are bounded between [0,1] for simplicity.

The goal is to keep the cumulative loss $\sum_{t=1}^{n} \ell(\mathbf{x}_t;\, \mathbf{a}_t, b_t)$ close to the best fixed vector chosen, $\min_{\mathbf{x}\in\mathbb{R}^d} \sum_{t=1}^{n} \ell(\mathbf{x};\, \mathbf{a}_t, b_t)$.

In the online setting, data $(\mathbf{a}_t, b_t)$ arrives one by one. If you simply stored all past data to make the next prediction, the amount of data would grow linearly with time t. Recomputing the best model $\mathbf{x}_t$ from scratch using all past t-1 data points becomes very slow and memory-intensive as t (and potentially n) gets large.

With our general reduction framework, we show an $\epsilon$-regret upper bound as follows.

**Theorem 4.** *For any $\epsilon \in (0, 1)$, Algorithm 6 has regret -*

$$\mathrm{Regret}_\epsilon(n) \;=\; O\big(\epsilon^{-2}\, d\, \log n\;\log(\epsilon^{-1} dn)\big).$$

### Method and results

Similar to the low-rank matrix approximation problem, we utilize the leverages core method to learn a subspace that preserves information regarding the regression. Specifically, we use the leverage score to learn a $\epsilon$-subspace embedding, which is defined as follows.
Instead of recomputing the least-squares solution on all past data at each step, we maintain a small summary that preserves all least-squares costs up to a $(1 \pm \epsilon)$ factor:

### Subspace Embedding :

- The algorithm constructs a small matrix $S^t$. (See Definition-5 for the embedding property: applying $S^t$ such that it should not distort the core structure of the regression problem much.)

- This matrix $S^t$ is built by sampling rows from the past data. It samples rows based on their leverage scores—more important rows are more likely to be included in the S (see Algorithm 5, steps 7–9).
- The size of this S, $m$, is much smaller than the number of past data points $(t-1)$. Its size depends on the desired accuracy $\epsilon$ and the data dimension $d$, but not directly on $t$ (see $m$ in Theorem 5 and Algorithm 5, step 1).

**Definition 4** ($\epsilon$ - Subspace Embedding)**.** *A matrix $S \in \mathbb{R}^{m\times n}$ is an $\epsilon$subspace embedding for $A \in \mathbb{R}^{(n)\times d}$ if for all $\mathbf{x} \in \mathbb{R}^d$,*

$$(1-\epsilon)\,\|A\mathbf{x}\|_2 \;\leq\; \|\,S\,A\,\mathbf{x}\|_2 \;\leq\; (1+\epsilon)\,\|A\mathbf{x}\|_2.$$

The subspace embedding serves the same functionality as coreset in the problem of online regression, it preserves the loss of information while enjoying a much lower dimension. In the online regression problem context, we define the leverage score as follows.

**Leverage Scores :**

Imagine you have a lot of past data points ($a_i, b_i$ for i < t). Some data points might be more important or influential to show the best prediction model. Leverage scores are a mathematical way to quantify this importance for each past data point i. Higher score means more important. (Below definition gives the formal definition using Singular Value Decomposition).

**Definition 5** (Leverage Score)**.** *Let $A = U\Sigma V^\top$ be its SVD. For $i\in [n], i^{th}$ leverage score of $A$ is*

$$\tau_i = \|U_{i,*}\|_2^2$$

With the leverage score, we present the Algorithm. The algorithm follows the general transformation framework, where the regression problem is solved at every step with the sketch derived from the aggregated matrix using leverage score. For notational convenience, we construct the sketch by appending rows instead of columns as we did in above section.

**Algorithm: Online Regression via Leverage-Score Sketching**

**Algorithm 6** Online consistent regression

**Require:** Approximation parameter $\epsilon \in (0, 1)$.

1: Set $\delta = O\big(\epsilon/n\big)$ and $m = O\big(\epsilon^{-2}\, d\, \log(\delta^{-1} d)\big)$.
2: **for** $t = 1, \ldots, n$ **do**
3: Construct $A_{t-1} \in \mathbb{R}^{(t-1)\times d}$ by stacking $a_1^\top, \ldots, a_{t-1}^\top$.
4: Construct $b \in \mathbb{R}^{t-1}$ by stacking $b_1, \ldots, b_{t-1}$.
5: Initialize $S^t \in \mathbb{R}^{m\times(t-1)}$ to the zero matrix.
6: **for** $j = 1, \ldots, m$ **do**
7: Sample $i \in [t-1]$ with probability $p_i \;=\; \frac{\tau_i(A_{t-1})}{\sum_{j=1}^{t-1} \tau_j(A_{t-1})}$ .
8: Sample $w \in \mathbb{R}$ uniformly from $\left[\frac{1}{\sqrt{m\, p_i}}\;,\; \frac{1+\epsilon}{\sqrt{m\, p_i}}\right]$.
9: Replace the $j$-th row of $S^t$ with $w\, e_i^\top$, where $e_i \in \mathbb{R}^{t-1}$ is the $i$-th standard basis vector.
10: Solve $x_t \;=\; \arg\min_x \left\| S^t A_{t-1}\, x \;-\; S^t b \right\|_2$
(e.g. by an iterative method such as Newton's method).
11: Receive $a_t \in \mathbb{R}^d$, $b_t \in \mathbb{R}$ and incur loss $\ell(x_t, a_t, b_t)$.

**Explanation *(Algorithm 6)* :**

1. Set up parameters $(\epsilon, m)$.
2. For each time step $t = 1$ to $n$:
3. Consider all past data $A_{t-1}$, $b_{t-1}$.
4. Calculate leverage scores for past data points (or use existing scores).
5. Build the small sketching matrix $S^t$ by sampling $m$ times from the past $t-1$ data points, prioritizing those with high leverage scores and then rescaling them (steps 7-9).
   - Calculate importance scores (leverage scores $\tau_i$) for each past data point.
   - Define probabilities $p_i = \tau_i / \sum_{k=1}^{t-1} \tau_k$.
   - Sample index $i$ from $\{1, \ldots, t-1\}$ according to these probabilities $p_i$. (More important past points are picked more often).
   - Sample a random weight $w$ uniformly from the range $\left[\frac{1}{\sqrt{mp_i}}, \frac{1+\epsilon}{\sqrt{mp_i}}\right]$.
   - Set the $j$-th row of $S^t$ to be a vector with $w$ at the $i$-th position and zeros elsewhere ($we_i^T$).

6. Solve a *small* regression problem: find $x_t$ that minimizes $\|S^t A_{t-1} x - S^t b_{t-1}\|_2$ (step 10). This is computationally cheaper because $S^t A_{t-1}$ and $S^t b_{t-1}$ are small.
7. Receive the actual $a_t$, $b_t$ for the current round.
8. Calculate and record the loss for this round using $x_t$.
9. Add $a_t$, $b_t$ to the history for the next step.

**Theorem 5.** *For any $\epsilon, \delta \in (0,1)$, if*

$$m \;=\; O\big(\epsilon^{-2}\, d\, \log(\delta^{-1} d)\big),$$

*then with probability at least $1-\delta$, the $S^t \in \mathbb{R}^{m\times(t-1)}$ is an $\epsilon$–subspace embedding for $A_{t-1}$ with*

$$O\big(\epsilon^{-2}\, d\, \log(\delta^{-1} d)\big)$$

*rows (i.e. columns of the $S^t$).*

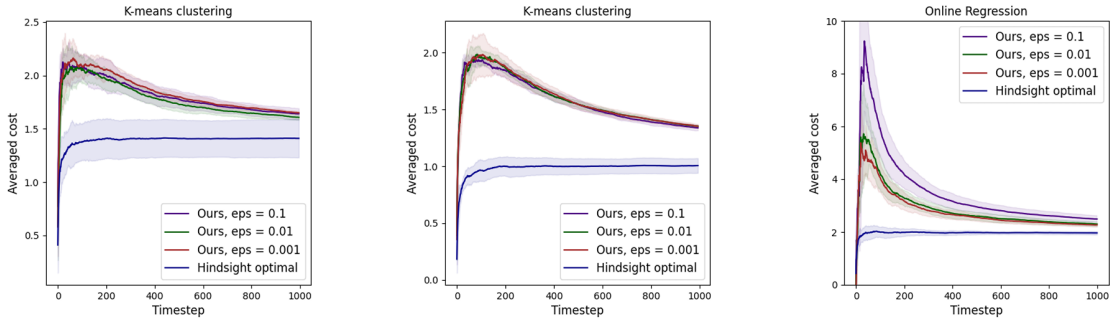
To obtain the Theorem 4, we proceed in two steps:

1. We first bound the average sensitivity of the leverage-score sampling procedure.
2. We then invoke Theorem 5 together with the general batch-to-online reduction (Theorem 1).

Combining these gives the claimed $\epsilon$–approximate regret bound for online regression.

# 6 Experiments

Experiments were conducted for online $k$-means clustering and online regression problems. The performance of the proposed algorithm is compared with the hindsight optimal solution. For $k$-means clustering, the hindsight optimal solution is obtained by applying $k$-means++ to all the data. For regression, the least square formula is used to compute the hindsight optimal solution.




**(a)** $k$-means, 3 clusters **(b)** $k$-means, 5 clusters **(c)** Regression

Figure 1: The experiments were conducted for $\epsilon = 0.1$, 0.01 and 0.001, and with $k = 3$ or $k = 5$ clusters.

Experiments with online k-means and regression showed that the proposed method works well in practice. The main performance metric used was the averaged cost over time, which is the total loss of the online algorithm up to each point, divided by the number of steps. The researchers compared this averaged cost for their online algorithms (with different values of $\varepsilon$ for accuracy) to the best possible offline solution. They observed that as $\varepsilon$ became smaller (indicating higher accuracy), the online algorithm's performance got closer to the optimal offline one. This shows that the method has low regret and confirms that the batch-to-online coreset approach is effective in real-world scenarios.

# 7 Conclusion

We have learned how to adapt and devise an algorithm for online setting of some problems where we don't have all the data in hindsight, rather it arrives point by point with time. The high level idea is to make a small summary("coreset") of the data points seen so far at every step and use it to perform almost as well as if we had the whole data from the start.